

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ №1
Дисциплина: «Оценка сложности алгоритмов»

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Яхонтов А.А.

Краснодар
2026

Тема. Деревья.

Цель. Написать рекурсивную процедуру, вычисляющую разность между количеством вершин, имеющих только левых потомков, и вершин, имеющих только правых потомков, в бинарном упорядоченном дереве. Найти функции сложности алгоритма (рекурсивную и не рекурсивную). Произвести экспериментальные расчёты, подтверждающие правильность аналитических результатов.

Реализуем бинарное упорядоченное дерево:

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def insert_node(root, value):
    if root is None:
        return TreeNode(value)

    if value < root.value:
        root.left = insert_node(root.left, value)
    else:
        root.right = insert_node(root.right, value)
    return root

def generate_tree(values):
    if not values:
        return None

    root = TreeNode(values[0])
    for val in values[1:]:
        insert_node(root, val)
    return root
```

Теперь реализуем метод, рекурсивно вычисляющий разность между количеством вершин, имеющих только левых потомков, и вершин, имеющих только правых потомков, в дереве:

```
def recursion_count(node):
    if node is None:
        return 0
    elif (node.left is not None) and (node.right is None):
        return 1 + recursion_count(node.left) + recursion_count(node.right)
    elif (node.left is None) and (node.right is not None):
        return -1 + recursion_count(node.left) + recursion_count(node.right)
    else:
        return recursion_count(node.left) + recursion_count(node.right)
```

Временная сложность:

$$T(n) = 1 + T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) = O(n) \text{ — функция сложности линейная}$$

Ёмкостная сложность:

$$S(n) = n = O(n) \text{ — функция сложности линейная}$$

Реализуем метод, итеративно вычисляющий разность между количеством вершин, имеющих только левых потомков, и вершин, имеющих только правых потомков, в дереве:

```
def iteration_count(root):
    if root is None:
        return 0

    count = 0
    stack = [root]

    while stack:
        node = stack.pop()

        if (node.left is not None) and (node.right is None):
            count += 1
        elif (node.left is None) and (node.right is not None):
            count -= 1

        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)

    return count
```

Временная сложность:

$$T(n) = 6n + C = O(n) \text{ — функция сложности линейная}$$

Ёмкостная сложность:

$$S(n) = \log_2 n = O(\log n) \text{ — функция сложности логарифмическая}$$

